Viva Question Explanation

CpE 211 Laboratory — Lab 2: Linear Search (Lab_02_Linear_Search)

Student	Kane Paul A. Mari
Student No.	921691
Section	BSCpE-2
Instructor	Sir Jamie Eduardo Rosal
Date	10/23/2026

> *Question: Why does Linear Search work on unsorted data, and when should it be avoided?*

1. Why Linear Search Works on Unsorted Data

Linear search works on unsorted data because it does not depend on any ordering — it checks each element one by one. Starting at the first element, it compares every item against the target and moves on to the next until the target is found or the list ends. At no point does it assume the elements are arranged in any particular order, so whether the array is sorted, unsorted, or completely random, the algorithm behaves exactly the same way.

Its correctness comes from being exhaustive: since no element is skipped, a target that exists cannot be missed, and its position turns out to be irrelevant. This is exactly what the lab program demonstrates — *LibraryFinder.findFirst()* scans the titles array from index 0 forward, and *countMatches()* walks the entire array, and both produce correct results whether or not the elements are in order.

2. When Linear Search Should Be Avoided

Linear search should be avoided on very large datasets, because it goes through every element and compares it. Its running time grows directly with the size of the list: in the worst case — when the target is missing or sits at the very end — every item is examined. Let's say a list has 1,000,000 items; the worst-case scenario would make 1,000,000 comparisons, which wastes an enormous amount of time as the data grows.

For large collections, a sorted structure searched with binary search is far more efficient: each comparison halves the search range, so finding an item among 1,000,000 entries takes only about 20 comparisons instead of up to a million. In short, linear search is perfectly reasonable for small or unsorted collections, and for tasks like finding the first occurrence of a title or counting matches (exactly this lab's use case), but it should be avoided when the dataset is very large and lookup speed matters.